

DESIGN AND ANALYSIS OF ALGORITHMS

DAY 5 – DYNAMIC PROGRAMMING – OPTIMIZATION PROBLEMS

Name: N. PARNITA

Register No.: 192525322

Course code: CSA0617

Course Name: Design and Analysis of Algorithm

TABLE OF CONTENTS

Experiment 1: 0/1 Knapsack Problem

Experiment 2: Travelling Salesperson Problem using Dynamic Programming

Experiment 3: Optimal Binary Search Tree

Experiment 4: 0/1 Knapsack Problem – Alternative Implementation

Experiment 5: 0/1 Knapsack using Memoization

Experiment 6: Subset Sum Problem

Experiment 7: Travelling Salesperson – Memoization

Experiment 8: Travelling Salesperson using DP Table

Experiment 9: Optimal Binary Search Tree – Frequency Based

Experiment 10: Matrix Chain Multiplication

Experiment 11: Coin Change – Minimum Coins

Experiment 12: Rod Cutting Problem

Experiment 13: Minimum Cost Path

Experiment 1: 0/1 Knapsack Problem

Objective: To maximize profit by selecting items whose total weight does not exceed the given capacity.

Problem Statement: To maximize profit by selecting items whose total weight does not exceed the given capacity.

Sample Input: weights = [1,3,4,5], values = [1,4,5,7], capacity = 7

Sample Output: Maximum Profit: 9

Time Complexity: $O(n \times W)$ time and $O(n \times W)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def knapsack(weights, values, capacity):
    n = len(weights)
    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(
                    values[i - 1] + dp[i - 1][w - weights[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][capacity]

weights = list(map(int, input("Enter weights: ").split()))
values = list(map(int, input("Enter values: ").split()))
capacity = int(input("Enter capacity: "))

print("Maximum Profit:", knapsack(weights, values, capacity))
```

Execution / Output

```
Enter weights: 1 3 4 5
Enter values: 1 4 5 7
Enter capacity: 7
Maximum Profit: 9
```

Result: The program was implemented and the expected result was obtained.

Experiment 2: Travelling Salesperson Problem using Dynamic Programming

Objective: To find the minimum cost tour that visits every city exactly once and returns to the starting city.

Problem Statement: To find the minimum cost tour that visits every city exactly once and returns to the starting city.

Sample Input: graph = [[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]

Sample Output: Minimum Cost: 80

Time Complexity: $O(n^2 \times 2^n)$ time and $O(n \times 2^n)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
from functools import lru_cache

def tsp(graph):
    n = len(graph)

    @lru_cache(None)
    def visit(mask, pos):
        if mask == (1 << n) - 1:
            return graph[pos][0]

        ans = float("inf")
        for city in range(n):
            if mask & (1 << city) == 0:
                ans = min(
                    ans,
                    graph[pos][city] +
                    visit(mask | (1 << city), city)
                )
        return ans

    return visit(1, 0)

graph = []
n = int(input("Enter number of cities: "))
print("Enter the cost matrix:")
for _ in range(n):
    graph.append(list(map(int, input().split())))

print("Minimum Cost:", tsp(graph))
```

Execution / Output

```
Enter number of cities: 4
Enter the cost matrix:
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
```

Minimum Cost: 80

Result: The program was implemented and the expected result was obtained.

Experiment 3: Optimal Binary Search Tree

Objective: To find the minimum search cost for constructing an optimal binary search tree from keys with given frequencies.

Problem Statement: To find the minimum search cost for constructing an optimal binary search tree from keys with given frequencies.

Sample Input: keys = [10,20,30], frequencies = [34,8,50]

Sample Output: Optimal BST Cost: 142

Time Complexity: $O(n^3)$ time and $O(n^2)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def optimal_bst(keys, freq):
    n = len(keys)
    cost = [[0 for _ in range(n)] for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1
            cost[i][j] = float("inf")
            total_freq = sum(freq[i:j + 1])

            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0
                cost[i][j] = min(
                    cost[i][j],
                    left + right + total_freq
                )

    return cost[0][n - 1]

keys = list(map(int, input("Enter keys: ").split()))
freq = list(map(int, input("Enter frequencies: ").split()))

print("Optimal BST Cost:", optimal_bst(keys, freq))
```

Execution / Output

```
Enter keys: 10 20 30
Enter frequencies: 34 8 50
Optimal BST Cost: 142
```

Result: The program was implemented and the expected result was obtained.

Experiment 4: 0/1 Knapsack Problem – Alternative Implementation

Objective: To solve the 0/1 Knapsack problem using bottom-up dynamic programming.

Problem Statement: To solve the 0/1 Knapsack problem using bottom-up dynamic programming.

Sample Input: weights = [1,3,4,5], values = [1,4,5,7], capacity = 7

Sample Output: Maximum Profit = 9

Time Complexity: $O(n \times W)$ time and $O(n \times W)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def knapsack(wt, val, W):
    n = len(wt)
    dp = [[0] * (W + 1) for _ in range(n + 1)]

    for i in range(1, n + 1):
        for w in range(W + 1):
            if wt[i - 1] <= w:
                dp[i][w] = max(
                    val[i - 1] + dp[i - 1][w - wt[i - 1]],
                    dp[i - 1][w]
                )
            else:
                dp[i][w] = dp[i - 1][w]

    return dp[n][W]

wt = list(map(int, input("Enter weights: ").split()))
val = list(map(int, input("Enter values: ").split()))
W = int(input("Enter capacity: "))

print("Maximum Profit =", knapsack(wt, val, W))
```

Execution / Output

```
Enter weights: 1 3 4 5
Enter values: 1 4 5 7
Enter capacity: 7
Maximum Profit = 9
```

Result: The program was implemented and the expected result was obtained.

Experiment 5: 0/1 Knapsack using Memoization

Objective: To solve the 0/1 Knapsack problem using recursive dynamic programming with memoization.

Problem Statement: To solve the 0/1 Knapsack problem using recursive dynamic programming with memoization.

Sample Input: weights = [1,3,4,5], values = [1,4,5,7], capacity = 7

Sample Output: Maximum Profit = 9

Time Complexity: $O(n \times W)$ time and $O(n \times W)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def knapsack(W, wt, val, n, memo):
    if n == 0 or W == 0:
        return 0

    if memo[n][W] != -1:
        return memo[n][W]

    if wt[n - 1] <= W:
        memo[n][W] = max(
            val[n - 1] + knapsack(W - wt[n - 1], wt, val, n - 1, memo),
            knapsack(W, wt, val, n - 1, memo)
        )
    else:
        memo[n][W] = knapsack(W, wt, val, n - 1, memo)

    return memo[n][W]

wt = list(map(int, input("Enter weights: ").split()))
val = list(map(int, input("Enter values: ").split()))
W = int(input("Enter capacity: "))

memo = [[-1] * (W + 1) for _ in range(len(wt) + 1)]
print("Maximum Profit =", knapsack(W, wt, val, len(wt), memo))
```

Execution / Output

```
Enter weights: 1 3 4 5
Enter values: 1 4 5 7
Enter capacity: 7
Maximum Profit = 9
```

Result: The program was implemented and the expected result was obtained.

Experiment 6: Subset Sum Problem

Objective: To determine whether a subset of the given array has a sum equal to the target.

Problem Statement: To determine whether a subset of the given array has a sum equal to the target.

Sample Input: arr = [3,34,4,12,5,2], target = 9

Sample Output: True

Time Complexity: $O(n \times S)$ time and $O(n \times S)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def subset_sum(arr, target):
    n = len(arr)
    dp = [[False] * (target + 1) for _ in range(n + 1)]

    for i in range(n + 1):
        dp[i][0] = True

    for i in range(1, n + 1):
        for j in range(1, target + 1):
            if arr[i - 1] <= j:
                dp[i][j] = (
                    dp[i - 1][j] or
                    dp[i - 1][j - arr[i - 1]]
                )
            else:
                dp[i][j] = dp[i - 1][j]

    return dp[n][target]

arr = list(map(int, input("Enter elements: ").split()))
target = int(input("Enter target: "))

print(subset_sum(arr, target))
```

Execution / Output

```
Enter elements: 3 34 4 12 5 2
Enter target: 9
True
```

Result: The program was implemented and the expected result was obtained.

Experiment 7: Travelling Salesperson – Memoization

Objective: To find the minimum travelling cost using memoized dynamic programming.

Problem Statement: To find the minimum travelling cost using memoized dynamic programming.

Sample Input: graph = [[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]

Sample Output: Minimum Cost = 80

Time Complexity: $O(n^2 \times 2^n)$ time and $O(n \times 2^n)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
from functools import lru_cache

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)

@lru_cache(None)
def tsp(mask, pos):
    if mask == (1 << n) - 1:
        return graph[pos][0]

    ans = float("inf")

    for city in range(n):
        if not mask & (1 << city):
            ans = min(
                ans,
                graph[pos][city] +
                tsp(mask | (1 << city), city)
            )

    return ans

print("Minimum Cost =", tsp(1, 0))
```

Execution / Output

```
Minimum Cost = 80
```

Result: The program was implemented and the expected result was obtained.

Experiment 8: Travelling Salesperson using DP Table

Objective: To solve the Travelling Salesperson Problem using an explicit dynamic programming table.

Problem Statement: To solve the Travelling Salesperson Problem using an explicit dynamic programming table.

Sample Input: graph = [[0,10,15,20],[10,0,35,25],[15,35,0,30],[20,25,30,0]]

Sample Output: Minimum Cost = 80

Time Complexity: $O(n^2 \times 2^n)$ time and $O(n \times 2^n)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def tsp_dp(graph):
    n = len(graph)
    dp = [[float("inf")] * n for _ in range(1 << n)]
    dp[1][0] = 0

    for mask in range(1 << n):
        for u in range(n):
            if mask & (1 << u):
                for v in range(n):
                    if not mask & (1 << v):
                        dp[mask | (1 << v)][v] = min(
                            dp[mask | (1 << v)][v],
                            dp[mask][u] + graph[u][v]
                        )
    ans = float("inf")
    for i in range(1, n):
        ans = min(
            ans,
            dp[(1 << n) - 1][i] + graph[i][0]
        )
    return ans

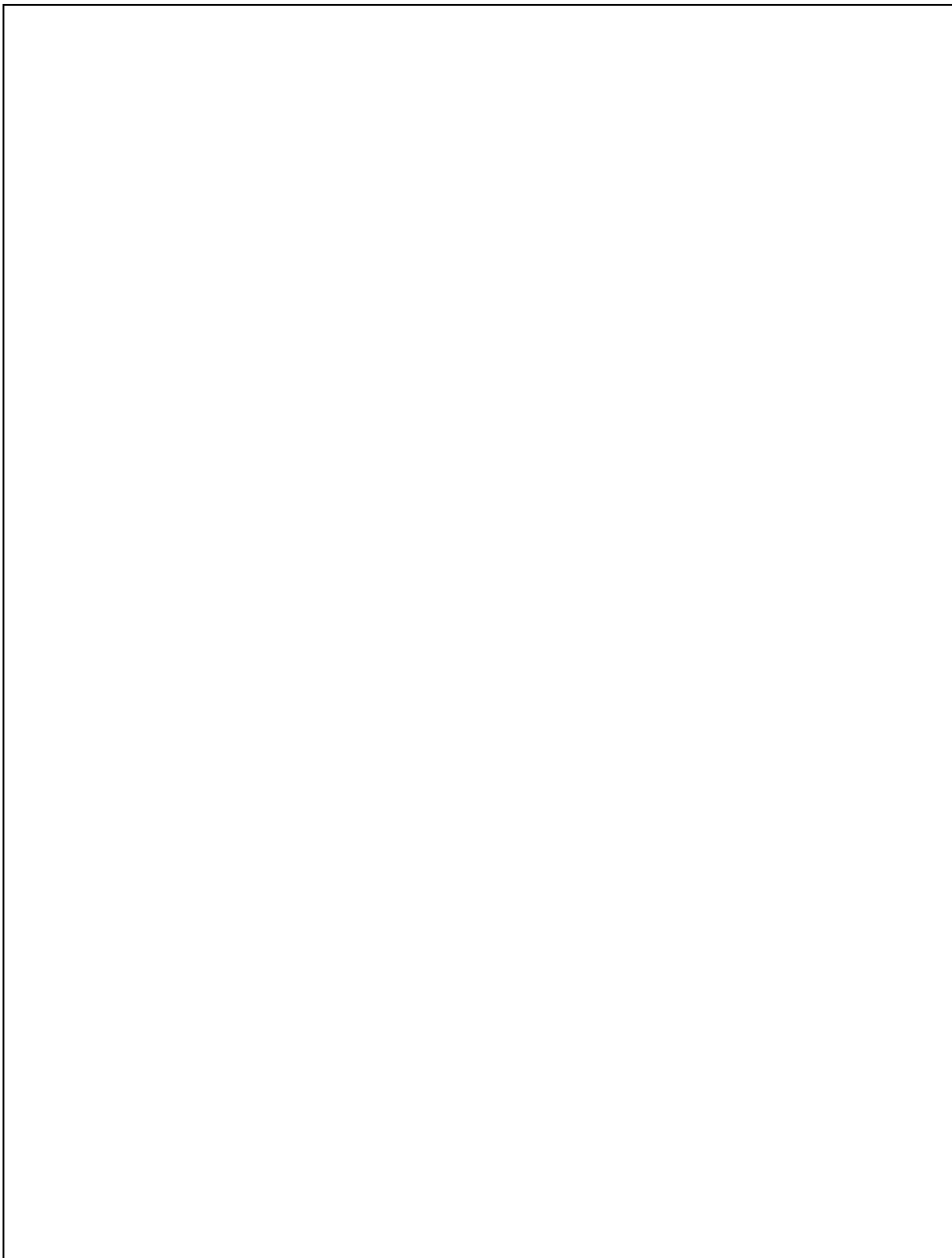
n = int(input("Enter number of cities: "))
graph = [list(map(int, input().split())) for _ in range(n)]

print("Minimum Cost =", tsp_dp(graph))
```

Execution / Output

```
Enter number of cities: 4
0 10 15 20
10 0 35 25
15 35 0 30
20 25 30 0
Minimum Cost = 80
```

Result: The program was implemented and the expected result was obtained.



Experiment 9: Optimal Binary Search Tree – Frequency Based

Objective: To find the minimum cost of an optimal binary search tree using only the frequency values.

Problem Statement: To find the minimum cost of an optimal binary search tree using only the frequency values.

Sample Input: frequencies = [34,8,50]

Sample Output: Optimal Cost = 142

Time Complexity: $O(n^3)$ time and $O(n^2)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def optimal_bst(freq):
    n = len(freq)
    cost = [[0] * n for _ in range(n)]

    for i in range(n):
        cost[i][i] = freq[i]

    for L in range(2, n + 1):
        for i in range(n - L + 1):
            j = i + L - 1
            cost[i][j] = float("inf")
            total = sum(freq[i:j + 1])

            for r in range(i, j + 1):
                left = cost[i][r - 1] if r > i else 0
                right = cost[r + 1][j] if r < j else 0

                cost[i][j] = min(
                    cost[i][j],
                    left + right + total
                )

    return cost[0][n - 1]

freq = list(map(int, input("Enter frequencies: ").split()))
print("Optimal Cost =", optimal_bst(freq))
```

Execution / Output

```
Enter frequencies: 34 8 50
Optimal Cost = 142
```

Result: The program was implemented and the expected result was obtained.

Experiment 10: Matrix Chain Multiplication

Objective: To determine the minimum number of scalar multiplications needed to multiply a chain of matrices.

Problem Statement: To determine the minimum number of scalar multiplications needed to multiply a chain of matrices.

Sample Input: arr = [10,20,30,40,30]

Sample Output: Minimum Multiplications = 30000

Time Complexity: $O(n^3)$ time and $O(n^2)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def matrix_chain(arr):
    n = len(arr)
    dp = [[0] * n for _ in range(n)]

    for L in range(2, n):
        for i in range(1, n - L + 1):
            j = i + L - 1
            dp[i][j] = float("inf")

            for k in range(i, j):
                q = (
                    dp[i][k] +
                    dp[k + 1][j] +
                    arr[i - 1] * arr[k] * arr[j]
                )
                dp[i][j] = min(dp[i][j], q)

    return dp[1][n - 1]

arr = list(map(int, input("Enter dimensions: ").split()))
print("Minimum Multiplications =", matrix_chain(arr))
```

Execution / Output

```
Enter dimensions: 10 20 30 40 30
Minimum Multiplications = 30000
```

Result: The program was implemented and the expected result was obtained.

Experiment 11: Coin Change – Minimum Coins

Objective: To find the minimum number of coins required to make a given amount.

Problem Statement: To find the minimum number of coins required to make a given amount.

Sample Input: coins = [1,2,5], amount = 11

Sample Output: Minimum Coins = 3

Time Complexity: $O(\text{amount} \times \text{number of coins})$ time and $O(\text{amount})$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def min_coins(coins, amount):
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0

    for i in range(1, amount + 1):
        for c in coins:
            if c <= i:
                dp[i] = min(dp[i], dp[i - c] + 1)

    return dp[amount]

coins = list(map(int, input("Enter coin values: ").split()))
amount = int(input("Enter amount: "))

result = min_coins(coins, amount)

if result == float("inf"):
    print("Amount cannot be formed")
else:
    print("Minimum Coins =", result)
```

Execution / Output

```
Enter coin values: 1 2 5
Enter amount: 11
Minimum Coins = 3
```

Result: The program was implemented and the expected result was obtained.

Experiment 12: Rod Cutting Problem

Objective: To maximize revenue by cutting a rod of a given length into pieces with known prices.

Problem Statement: To maximize revenue by cutting a rod of a given length into pieces with known prices.

Sample Input: price = [1,5,8,9,10,17,17,20], n = 8

Sample Output: Maximum Revenue = 22

Time Complexity: $O(n^2)$ time and $O(n)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def rod_cut(price, n):
    dp = [0] * (n + 1)

    for i in range(1, n + 1):
        for j in range(i):
            dp[i] = max(
                dp[i],
                price[j] + dp[i - j - 1]
            )

    return dp[n]

price = list(map(int, input("Enter prices: ").split()))
n = int(input("Enter rod length: "))

print("Maximum Revenue =", rod_cut(price, n))
```

Execution / Output

```
Enter prices: 1 5 8 9 10 17 17 20
Enter rod length: 8
Maximum Revenue = 22
```

Result: The program was implemented and the expected result was obtained.

Experiment 13: Minimum Cost Path

Objective: To find the minimum cost path from the top-left to the bottom-right of a cost matrix, moving right or down.

Problem Statement: To find the minimum cost path from the top-left to the bottom-right of a cost matrix, moving right or down.

Sample Input: cost = [[1,3,1],[1,5,1],[4,2,1]]

Sample Output: Minimum Cost = 7

Time Complexity: $O(m \times n)$ time and $O(m \times n)$ space.

Explanation: Dynamic programming stores solutions to overlapping subproblems so that each required state is computed efficiently and reused.

Python Program

```
def min_cost(cost):
    m = len(cost)
    n = len(cost[0])
    dp = [[0] * n for _ in range(m)]
    dp[0][0] = cost[0][0]
    for i in range(1, m):
        dp[i][0] = dp[i - 1][0] + cost[i][0]

    for j in range(1, n):
        dp[0][j] = dp[0][j - 1] + cost[0][j]

    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = cost[i][j] + min(
                dp[i - 1][j],
                dp[i][j - 1]
            )
    return dp[m - 1][n - 1]

m = int(input("Enter number of rows: "))
n = int(input("Enter number of columns: "))
cost = []
for _ in range(m):
    cost.append(list(map(int, input().split())))

print("Minimum Cost =", min_cost(cost))
```

Execution / Output

```
Enter number of rows: 3
Enter number of columns: 3
1 3 1
1 5 1
4 2 1
Minimum Cost = 7
```

Result: The program was implemented and the expected result was obtained.